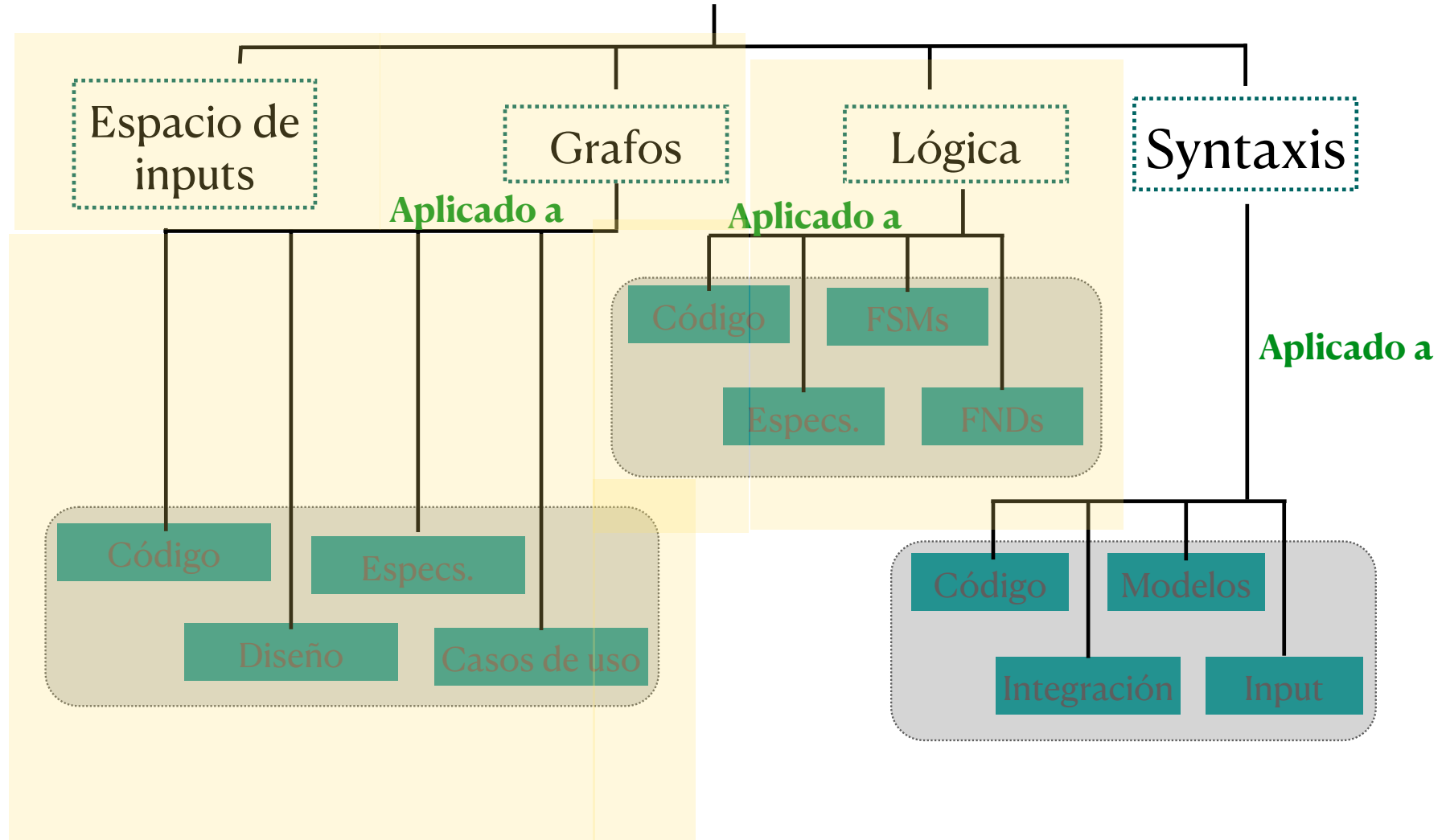


Testing basado en sintaxis

Valeria Bengolea

Estas transparencias están basadas en las desarrolladas por Ammann & Offutt como acompañamiento de su libro *Introduction to Software Testing (2nd Edition)* y por Manuel Núñez (Versión en castellano)

Cuatro estructuras para modelar software



Uso de sintaxis para generar tests

Muchos artefactos software siguen una reglas sintácticas estrictas.

La sintaxis se suele expresar cómo una gramática en un lenguaje como BNF.

Las descripciones sintácticas pueden provenir de muchos sitios:

- Programas.

- Descripción de inputs válidos.

Mutación: modificación en la sintaxis que tiene como objetivo generar tests (o inputs para tests) o sirve de guía par la generación de tests

Los tests se crean con dos objetivos en mente:

- Cubrir la sintaxis de una cierta manera.

- Violar la sintaxis (tests inválidos).

Cobertura para gramáticas

La Ingeniería del Software utiliza la teoría de autómatas de varias formas:

Lenguajes de programación definidos como BNF.

Comportamiento de programas descrito mediante FSMs.

Cobertura para gramáticas

La Ingeniería del Software utiliza la teoría de autómatas de varias formas:

Lenguajes de programación definidos como BNF.

Comportamiento de programas descrito mediante FSMs.

Inputs permitidos definidos mediante gramáticas.

Cobertura para gramáticas

Ejemplo de expresión regular:

$$(G s n \mid B t n)^*$$

*: Clausura cero o más veces
|: Elección

Genera cualquier secuencia que contenga $G s n$ y $B t n$.

G y B podrían representar comandos, métodos o eventos.

s , t y n pueden representar argumentos, parámetros o valores.

También podrían representar literales o conjuntos de valores.

Gramáticas BNF

Simbolo Inicial

NO Terminales

Producciones

Terminales



Gramáticas BNF

Simbolo Inicial

Stream ::= acción*

NO Terminales

acción ::= accG | accB

Producciones

accG ::= "G" s n

accB ::= "B" t n

s ::= dígito¹⁻³

t ::= dígito¹⁻³

n ::= dígito² "." dígito² "." dígito²

Terminales

dígito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Test basados en gramáticas

Test basados en gramáticas

Una cadena que satisface las reglas de derivación se dice que pertenece al lenguaje de la gramática.

Test basados en gramáticas

Una cadena que satisface las reglas de derivación se dice que pertenece al lenguaje de la gramática.

Para nuestro ejemplo, caso de test es una secuencia de cadenas que satisfacen la expresión regular.

Test basados en gramáticas

Una cadena que satisface las reglas de derivación se dice que pertenece al lenguaje de la gramática.

Para nuestro ejemplo, caso de test es una secuencia de cadenas que satisfacen la expresión regular.

Supongamos que s , t y n representan números. Entonces,

Test basados en gramáticas

Una cadena que satisface las reglas de derivación se dice que pertenece al lenguaje de la gramática.

Para nuestro ejemplo, caso de test es una secuencia de cadenas que satisfacen la expresión regular.

Supongamos que s, t y n representan números. Entonces,

G 26 080190

B 22 062794

G 22 112194

B 13 010903

Test basados en gramáticas

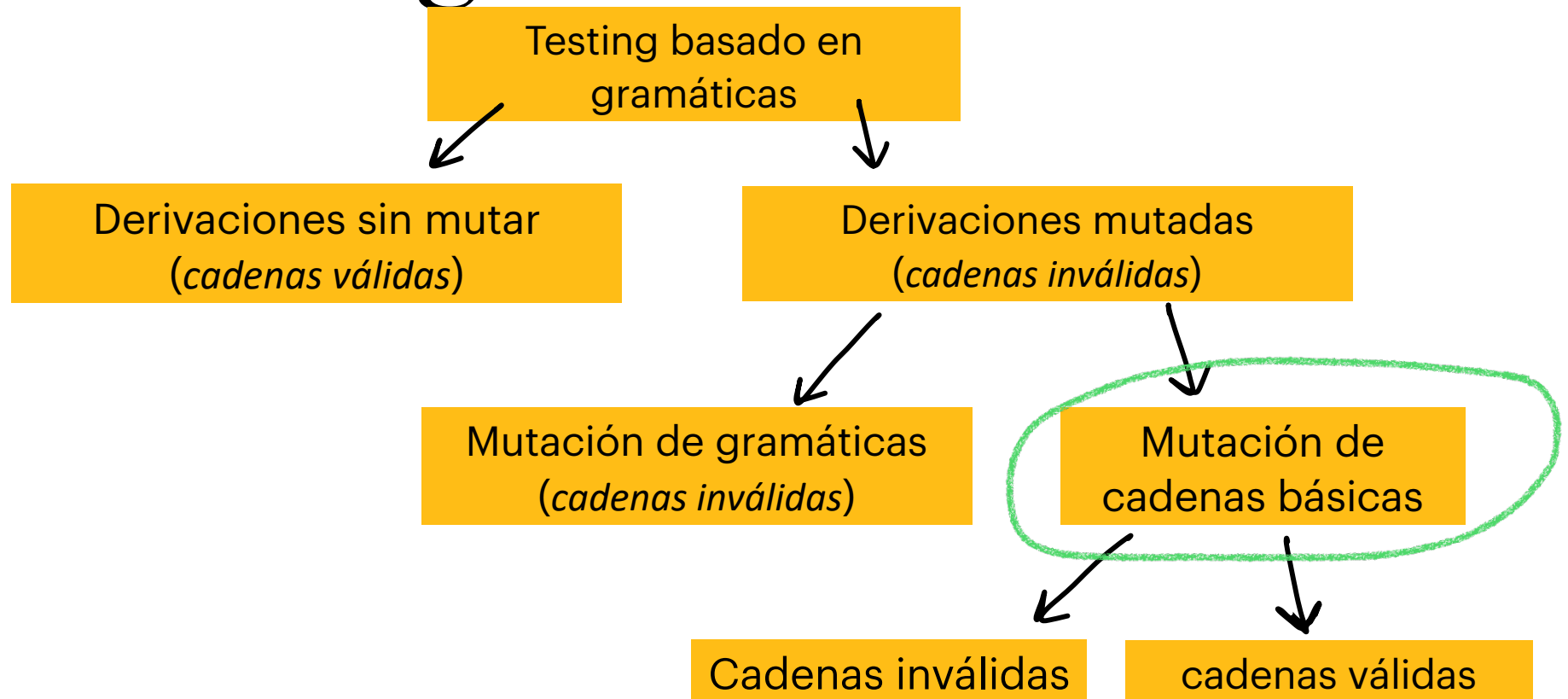
Test basados en gramáticas

- ★Reconocedor: ¿Está una cadena (o un test) en la gramática?
- ★Esto se llama parsear.
- ★Existen herramientas que facilitan la tarea de parsear.
- ★Los programas las pueden usar para validar inputs.

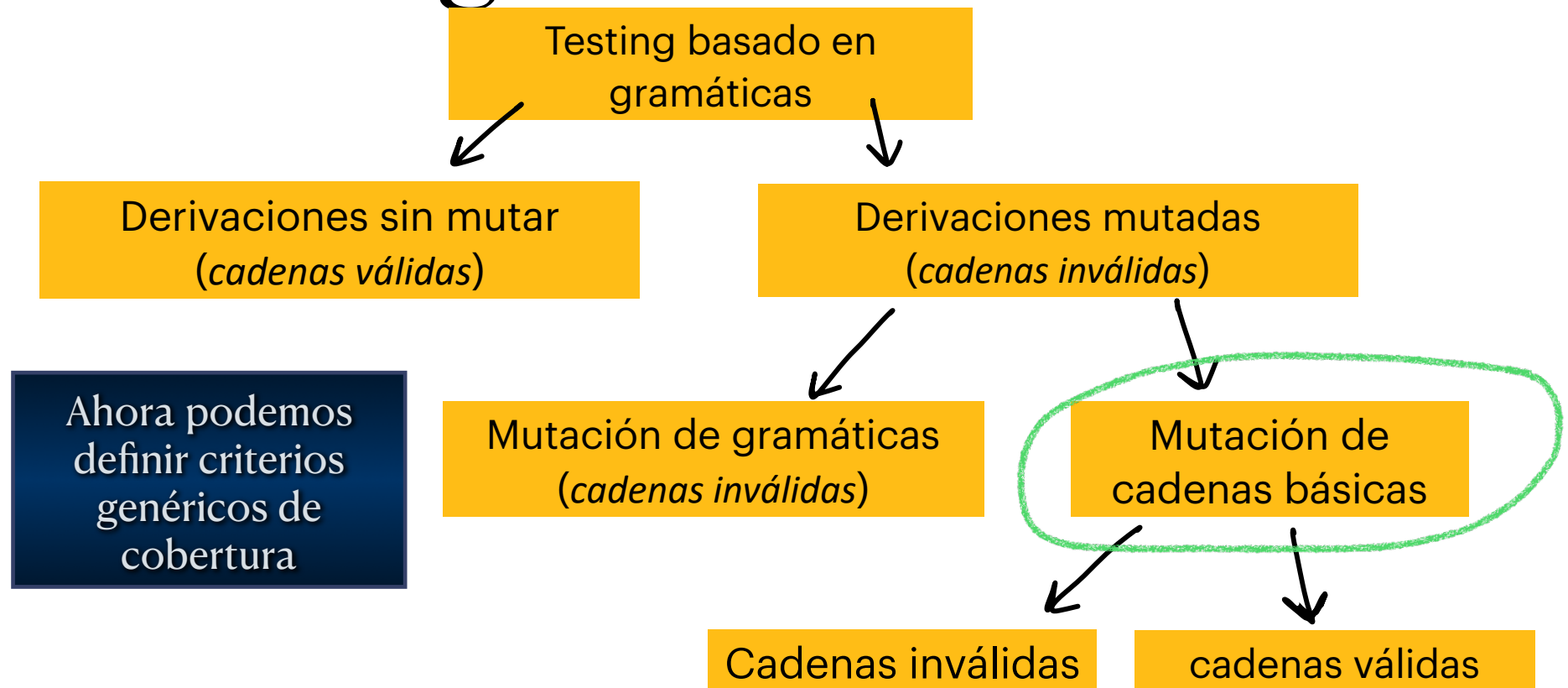
Test basados en gramáticas

- ★Reconocedor: ¿Está una cadena (o un test) en la gramática?
- ★Esto se llama parsear.
- ★Existen herramientas que facilitan la tarea de parsear.
- ★Los programas las pueden usar para validar inputs.
- ★Generador: Dada una gramática, deriva cadenas que están en la gramática.

Mutación como testing basado en gramáticas



Mutación como testing basado en gramáticas



Criterios de cobertura basados en gramáticas

El más común y sencillo: usar cada símbolo terminal y cada producción al menos una vez.

Criterios de cobertura basados en gramáticas

El más común y sencillo: usar cada símbolo terminal y cada producción al menos una vez.

Terminal Symbol Coverage (TSC): RT contiene cada símbolo terminal t de la gramática G .

Criterios de cobertura basados en gramáticas

El más común y sencillo: usar cada símbolo terminal y cada producción al menos una vez.

Terminal Symbol Coverage (TSC): RT contiene cada símbolo terminal t de la gramática G .

Production Coverage (PDC): RT contiene cada producción p de la gramática G .

Criterios de cobertura basados en gramáticas

El más común y sencillo: usar cada símbolo terminal y cada producción al menos una vez.

Terminal Symbol Coverage (TSC): RT contiene cada símbolo terminal t de la gramática G .

Production Coverage (PDC): RT contiene cada producción p de la gramática G .

PDC subsume TSC.

Criterios de cobertura basados en gramáticas

El más común y sencillo: usar cada símbolo terminal y cada producción al menos una vez.

Terminal Symbol Coverage (TSC): RT contiene cada símbolo terminal t de la gramática G .

Production Coverage (PDC): RT contiene cada producción p de la gramática G .

PDC subsume TSC.

Gramáticas y grafos son intercambiables: $PDS \approx EC$ y $TSC \approx NC$.

Criterios de cobertura basados en gramáticas

El más común y sencillo: usar cada símbolo terminal y cada producción al menos una vez.

Terminal Symbol Coverage (TSC): RT contiene cada símbolo terminal t de la gramática G .

Production Coverage (PDC): RT contiene cada producción p de la gramática G .

PDC subsume TSC.

Gramáticas y grafos son intercambiables: $PDS \approx EC$ y $TSC \approx NC$.

Se podrían definir otros criterios.....

Criterios de cobertura basados en gramáticas

Podríamos proponer derivar todas las cadenas posibles.

Derivation Coverage (DC): RT contiene todas las cadenas que están en la gramática G .

Criterios de cobertura basados en gramáticas

Podríamos proponer derivar todas las cadenas posibles.

Derivation Coverage (DC): RT contiene todas las cadenas que están en la gramática G .

El número de tests en TSC está acotado por el número de símbolos terminales (en el ejemplo de stream, tenemos 13).



Criterios de cobertura basados en gramáticas

Podríamos proponer derivar todas las cadenas posibles.

Derivation Coverage (DC): RT contiene todas las cadenas que están en la gramática G .

El número de tests en TSC está acotado por el número de símbolos terminales (en el ejemplo de stream, tenemos 13).

El número de tests en PDC está acotado por el número de reglas de producción (en el ejemplo de stream, tenemos 18).

Criterios de cobertura basados en gramáticas

Podríamos proponer derivar todas las cadenas posibles.

Derivation Coverage (DC): RT contiene todas las cadenas que están en la gramática G .

El número de tests en TSC está acotado por el número de símbolos terminales (en el ejemplo de stream, tenemos 13).

El número de tests en PDC está acotado por el número de reglas de producción (en el ejemplo de stream, tenemos 18).

El número de tests en DC depende de los detalles de la gramática (2.000.000.000) en el ejemplo de stream sin tener en cuenta la primera producción).

Criterios de cobertura basados en gramáticas

Podríamos proponer derivar todas las cadenas posibles.

Derivation Coverage (DC): RT contiene todas las cadenas que están en la gramática G .

El número de tests en TSC está acotado por el número de símbolos terminales (en el ejemplo de stream, tenemos 13).

El número de tests en PDC está acotado por el número de reglas de producción (en el ejemplo de stream, tenemos 18).

El número de tests en DC depende de los detalles de la gramática (2.000.000.000) en el ejemplo de stream sin tener en cuenta la primera producción).

Todos los tests que usamos en TSC, PDC o DC están en la gramática.... ¿Qué hacemos con los tests que NO están en la gramática?

Criterios de cobertura basados en gramáticas

Podríamos proponer derivar todas las cadenas posibles.

Derivation Coverage (DC): RT contiene todas las cadenas que están en la gramática G .

El número de tests en TSC está acotado por el número de símbolos terminales (en el ejemplo de stream, tenemos 13).

El número de tests en PDC está acotado por el número de reglas de producción (en el ejemplo de stream, tenemos 18).

El número de tests en DC depende de los detalles de la gramática (2.000.000.000) en el ejemplo de stream sin tener en cuenta la primera producción).

Todos los tests que usamos en TSC, PDC o DC están en la gramática.... ¿Qué hacemos con los tests que NO están en la gramática?

La respuesta es Mutation Testing.

Mutation testing

Las gramáticas describen cadenas válidas e inválidas.

Ambos tipos se pueden producir como mutantes.

Un mutante es una variación de una cadena válida. Nota: un mutante puede ser tanto una cadena válida como una inválida.

Las mutaciones se basan en operadores de mutación y en cadenas básicas (las que están en la gramática).

¿En que consiste la mutación?

¿En que consiste la mutación?

Visión general

Hacemos análisis **basado en mutaciones** siempre que

- Usamos reglas bien definidas,
- definidas mediante descripciones sintácticas,
- para realizar cambios sistemáticos,
- a la sintaxis o a los objetos derivados de la sintaxis.

¿En que consiste la mutación?

Visión general

Operadores de
mutación

Hacemos análisis **basado en mutaciones** siempre que

- Usamos **reglas** bien definidas,
- definidas mediante descripciones sintácticas,
- para realizar cambios sistemáticos,
- a la sintaxis o a los objetos derivados de la sintaxis.

¿En que consiste la mutación?

Visión general

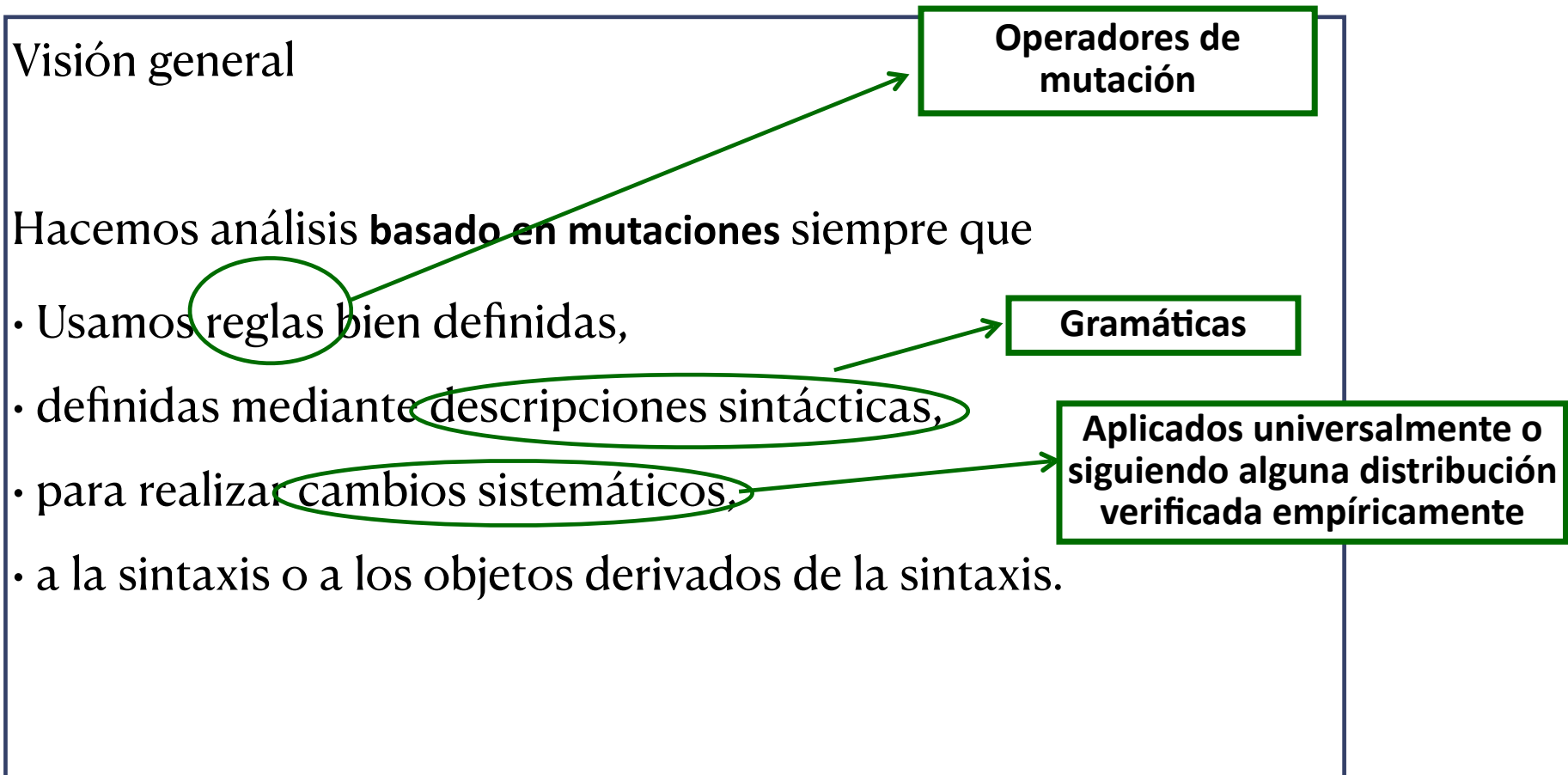
Operadores de
mutación

Hacemos análisis **basado en mutaciones** siempre que

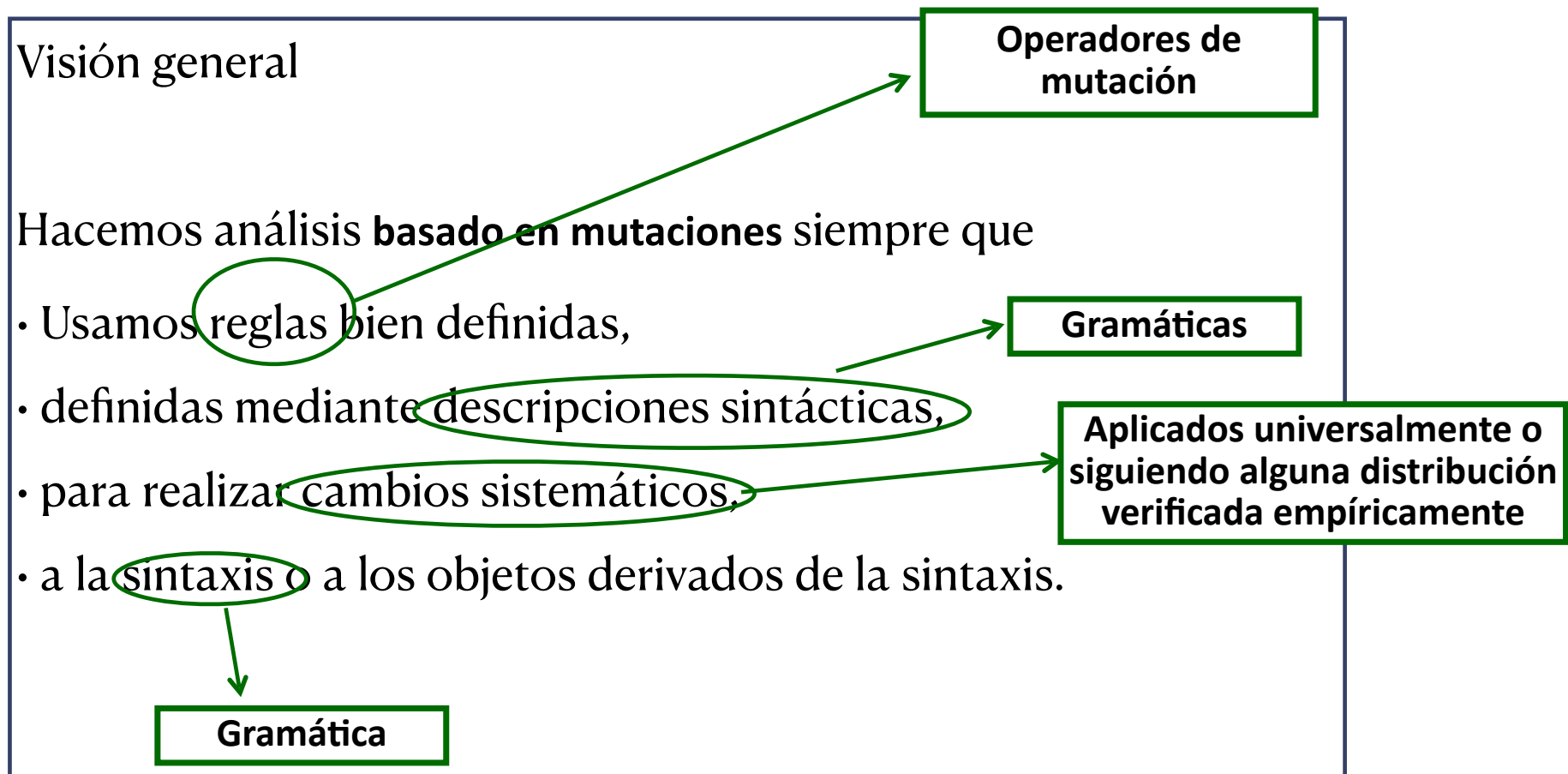
- Usamos **reglas** bien definidas,
- definidas mediante **descripciones sintácticas**,
- para realizar cambios sistemáticos,
- a la sintaxis o a los objetos derivados de la sintaxis.

Gramáticas

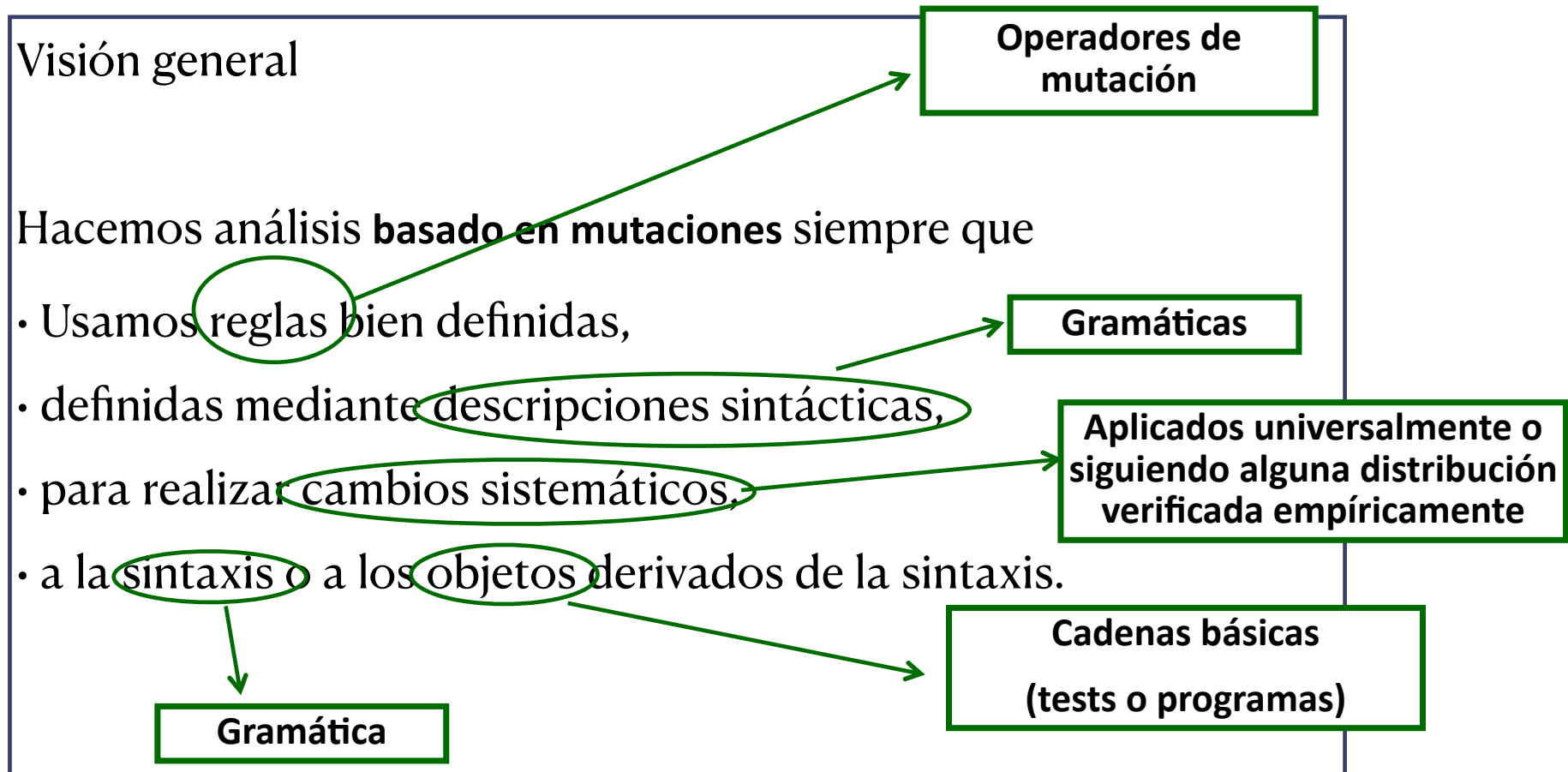
¿En que consiste la mutación?



¿En que consiste la mutación?



¿En que consiste la mutación?



Mutation testing

Cadenas básicas: las que están en la gramática.

Operadores de mutación: Una regla que especifica las variaciones sintácticas que se aplican a las cadenas anteriores.

Mutante: El resultado de la aplicación de un operador de mutación.

Cada mutante es o una cadena básica o se *parece mucho*.

Mutantes y cadenas de la gramática

La clave para el éxito de mutation testing es diseñar operadores de mutación: operadores bien diseñados dan lugar a testing muy potente.

Algunas veces las cadenas mutadas se basan en las cadenas básicas.

Otras veces se derivan directamente de la gramática.

- Las cadenas básicas se usan para generar test válidos.

- Los tests inválidos no necesitan estas cadenas.

Mutantes y cadenas de la gramática

Ejemplo:

Mutantes y cadenas de la gramática

Ejemplo:

<u>Mutantes válidos</u>	
<u>Cadenas básicas</u>	<u>Mutantes</u>
<i>G 26 08.01.90</i>	<i>B 26 08.01.90</i>
<i>B 22 06.27.94</i>	<i>B 45 06.27.94</i>

Mutantes y cadenas de la gramática

Ejemplo:

Mutantes válidos

Cadenas básicas

G 26 08.01.90

B 22 06.27.94

Mutantes

B 26 08.01.90

B 45 06.27.94

Mutantes inválidos

***7** 26 08.01.90*

B 22 06.27.1

Preguntas sobre Mutation

¿Deberíamos aplicar **más de un operador** al mismo tiempo?

Preguntas sobre Mutation

¿Deberíamos aplicar **más de un operador** al mismo tiempo?

En otras palabras, ¿debería una cadena mutada contener más de un elemento mutado?

Preguntas sobre Mutation

¿Deberíamos aplicar **más de un operador** al mismo tiempo?

En otras palabras, ¿debería una cadena mutada contener más de un elemento mutado?

Habitualmente, no: múltiples mutaciones pueden interferir entre ellas.

Preguntas sobre Mutation

¿Deberíamos aplicar **más de un operador** al mismo tiempo?

En otras palabras, ¿debería una cadena mutada contener más de un elemento mutado?

Habitualmente, no: múltiples mutaciones pueden interferir entre ellas.

De hecho, la experiencia dice que no....

Preguntas sobre Mutation

¿Deberíamos aplicar **más de un operador** al mismo tiempo?

En otras palabras, ¿debería una cadena mutada contener más de un elemento mutado?

Habitualmente, no: múltiples mutaciones pueden interferir entre ellas.

De hecho, la experiencia dice que no....

Aunque investigaciones recientes están encontrando excepciones.

Preguntas sobre Mutation

¿Deberíamos considerar todas las aplicaciones posibles de un operador de mutación?

Preguntas sobre Mutation

¿Deberíamos considerar todas las aplicaciones posibles de un operador de mutación?

Si, si estamos trabajando en un marco de mutación basada en programas.

Preguntas sobre Mutation

¿Deberíamos considerar todas las aplicaciones posibles de un operador de mutación?

Si, si estamos trabajando en un marco de mutación basada en programas.

¿Se han definido operadores de mutación para distintos lenguajes?

Preguntas sobre Mutation

¿Deberíamos considerar todas las aplicaciones posibles de un operador de mutación?

Si, si estamos trabajando en un marco de mutación basada en programas.

¿Se han definido operadores de mutación para distintos lenguajes?

Si, se ha definido para lenguajes de programación (Fortran, Lisp, Ada, C, C++, Java), lenguajes de especificación (SMV, Z, Object-Z, especificaciones algebraicas) y lenguajes de modelado (statecharts, diagramas de actividad).

Matando mutantes

Cuando se muta una cadena básica para crear cadenas válidas, esperamos que las mutaciones presenten un comportamiento distinto.



Matando mutantes

Cuando se muta una cadena básica para crear cadenas válidas, esperamos que las mutaciones presenten un comportamiento distinto.

Esta es la idea, en particular, cuando las gramáticas corresponden a lenguajes de programación, las cadenas son programas, y las cadenas básicas son programas que ya existen.



Matando mutantes

Cuando se muta una cadena básica para crear cadenas válidas, esperamos que las mutaciones presenten un comportamiento distinto.

Esta es la idea, en particular, cuando las gramáticas corresponden a lenguajes de programación, las cadenas son programas, y las cadenas básicas son programas que ya existen.

Matar mutantes: Dado un mutante $m \in M$ para una derivación D y un test t , decimos que t mata a m sii el resultado de aplicar t sobre D es diferente del resultado de aplicar t sobre m .

Matando mutantes

Cuando se muta una cadena básica para crear cadenas válidas, esperamos que las mutaciones presenten un comportamiento distinto.

Esta es la idea, en particular, cuando las gramáticas corresponden a lenguajes de programación, las cadenas son programas, y las cadenas básicas son programas que ya existen.

Matar mutantes: Dado un mutante $m \in M$ para una derivación D y un test t , decimos que t mata a m si el resultado de aplicar t sobre D es diferente del resultado de aplicar t sobre m .

La derivación D puede representarse o por una lista de producciones o por la cadena final.

Criterios de cobertura basados en sintaxis

Definimos cobertura en función de los mutantes que se maten.

Criterios de cobertura basados en sintaxis

Definimos cobertura en función de los mutantes que se maten.

Mutation Coverage (MC): Para cada $m \in M$, RT contiene exactamente un requisito: matar a m .

Criterios de cobertura basados en sintaxis

Definimos cobertura en función de los mutantes que se maten.

Mutation Coverage (MC): Para cada $m \in M$, RT contiene exactamente un requisito: matar a m .

Por tanto, la cobertura se equipara con el porcentaje de mutantes matados.

A este porcentaje se le llama mutation score.

Criterios de cobertura basados en sintaxis

Criterios de cobertura basados en sintaxis

Cuando queremos crear **strings inválidos**, sólo aplicamos los operadores de mutación

Esto resulta en dos criterios simples: usar cada operador una vez o aplicar el operador a cada producción una vez

Criterios de cobertura basados en sintaxis

Mutation Operator Coverage (MOC) : Para cada operador de mutación TR contiene exactamente un requerimiento: crear una cadena mutada m que se derive usando el operador.

Criterios de cobertura basados en sintaxis

Mutation Operator Coverage (MOC) : Para cada operador de mutación TR contiene exactamente un requerimiento: crear una cadena mutada m que se derive usando el operador.

```
Stream ::= acción*
acción ::= accG | accB
accG    ::= "G" s n
accB    ::= "B" t n
s       ::= dígito1-3
t       ::= dígito1-3
n       ::= dígito2 "." dígito2 "." dígito2
dígito  ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Criterios de cobertura basados en sintaxis

Mutation Operator Coverage (MOC) : Para cada operador de mutación TR contiene exactamente un requerimiento: crear una cadena mutada m que se derive usando el operador.

String básicos

G 25 08.01.90

B 21 06.27.94

Stream ::= acción*

acción ::= accG | accB

accG ::= "G" s n

accB ::= "B" t n

s ::= dígito¹⁻³

t ::= dígito¹⁻³

n ::= dígito² "." dígito² "." dígito²

dígito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Criterios de cobertura basados en sintaxis

Mutation Operator Coverage (MOC) : Para cada operador de mutación TR contiene exactamente un requerimiento: crear una cadena mutada m que se derive usando el operador.

String básicos

G 25 08.01.90

B 21 06.27.94

Mutation Operators

- Exchange actG and actB
- Replace digits with all other digits

Stream ::= acción*

acción ::= accG | accB

accG ::= "G" s n

accB ::= "B" t n

s ::= dígito¹⁻³

t ::= dígito¹⁻³

n ::= dígito² "." dígito² "." dígito²

dígito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Criterios de cobertura basados en sintaxis

Mutation Operator Coverage (MOC) : Para cada operador de mutación TR contiene exactamente un requerimiento: crear una cadena mutada m que se derive usando el operador.

String básicos

G 25 08.01.90

B 21 06.27.94

Mutation Operators

- Exchange actG and actB

- Replace digits with all other digits

Mutants usando MOC

B 25 08.01.90

B 23 06.27.94

Stream ::= acción*

acción ::= accG | accB

accG ::= "G" s n

accB ::= "B" t n

s ::= dígito¹⁻³

t ::= dígito¹⁻³

n ::= dígito² "." dígito² "." dígito²

dígito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Criterios de cobertura basados en sintaxis

Mutation Production Coverage (MPC) : Para cada operador de mutación TR contiene varios requerimientos: crear un string mutado m para cada producción que pueda ser mutada por el operador.

Criterios de cobertura basados en sintaxis

Mutation Production Coverage (MPC) : Para cada operador de mutación TR contiene varios requerimientos: crear un string mutado m para cada producción que pueda ser mutada por el operador.

Stream ::= acción*

acción ::= accG | accB

accG ::= "G" s n

accB ::= "B" t n

s ::= dígito¹⁻³

t ::= dígito¹⁻³

n ::= dígito² "." dígito² "." dígito²

dígito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Criterios de cobertura basados en sintaxis

Mutation Production Coverage (MPC) : Para cada operador de mutación TR contiene varios requerimientos: crear un string mutado m para cada producción que pueda ser mutada por el operador.

String básicos

G 25 08.01.90

B 21 06.27.94

Stream ::= acción*

acción ::= accG | accB

accG ::= "G" s n

accB ::= "B" t n

s ::= dígito¹⁻³

t ::= dígito¹⁻³

n ::= dígito² "." dígito² "." dígito²

dígito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Criterios de cobertura basados en sintaxis

Mutation Production Coverage (MPC) : Para cada operador de mutación TR contiene varios requerimientos: crear un string mutado m para cada producción que pueda ser mutada por el operador.

String básicos

G 25 08.01.90

B 21 06.27.94

Mutation Operators

- Exchange actG and actB
- Replace digits with all other digits

Stream ::= acción*

acción ::= accG | accB

accG ::= "G" s n

accB ::= "B" t n

s ::= dígito¹⁻³

t ::= dígito¹⁻³

n ::= dígito² "." dígito² "." dígito²

dígito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Criterios de cobertura basados en sintaxis

Mutation Production Coverage (MPC) : Para cada operador de mutación TR contiene varios requerimientos: crear un string mutado m para cada producción que pueda ser mutada por el operador.

String básicos

G 25 08.01.90

B 21 06.27.94

Mutation Operators

- Exchange actG and actB

- Replace digits with all other digits

Mutants using MPC

B 25 08.01.90 G 21 06.27.94

G 15 08.01.90 B 22 06.27.94

G 35 08.01.90 B 23 06.27.94

G 45 08.01.90 B 24 06.27.94

...

...

Stream ::= acción*

acción ::= accG | accB

accG ::= "G" s n

accB ::= "B" t n

s ::= dígito¹⁻³

t ::= dígito¹⁻³

n ::= dígito² "." dígito² "." dígito²

dígito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Mutation testing: reflexiones finales

El número de requisitos de testing para aplicar mutaciones depende de dos cosas:

- La sintaxis del artefacto que se muta

- Los operadores de mutación.

Es muy difícil aplicar mutation testing a mano.

Mutation testing es muy efectivo: se considera el “patrón oro” del testing.

Mutation testing se suele utilizar para evaluar la utilidad de otros criterios.

Durante el resto de este tema nos centramos en mutación en programas (uso habitual)